

SP-3a — Multi-Tracker SDK Foundation, RuTracker Decoupling, and RuTor POC

Status: Design (proposed) **Authored:** 2026-04-30 **Scope of this spec:** Spec 1 of the SP-3 program (foundation + RuTracker decoupling + RuTor POC). Spec 2 (innovations + anti-bluff audit + docs) is tracked separately.

Companion materials: docs/refactoring/decoupling/Lava_Multi_Tracker_SDK_Architecture_Plan.pdf (source-of-record research), docs/superpowers/specs/2026-04-28-sp2-go-api-migration-design.md (SP-2 sequencing context).

0. Decision Log

The seven brainstorming decisions that shape this spec, recorded for traceability:

#	Decision	Outcome
1	Scope decomposition	Two specs: SP-3a (this) covers Foundation + RuTracker decoupling + RuTor POC (~8.5 weeks). SP-3c..e (innovations + bluff audit + docs) is a separate program.
2	Decoupled Reusable Architecture rule	Hybrid extraction. Generic primitives (mirror manager, plugin registry, testing harness) live in vasic-digital/Tracker-SDK from day 1. Tracker contract (TrackerClient, capabilities, models) and RuTracker/RuTor implementations stay in this repo until RuTor validates the contract shape.
3	Sequencing vs SP-2 (Go API migration)	Kotlin-side first. Go-side rutracker refactor (PDF Phase 2 Tasks 2.7+2.8) becomes an out-of-spec SP-3a-bridge task that runs after SP-2's last release tag. SP-3a does not touch lava-api-go/internal/rutracker/.
4	Coverage target	"100% behavioral coverage with exemption ledger." Every public method of every new interface gets at least one real-stack test. Line-

#	Decision	Outcome
		coverage targets are secondary, gated at $\geq 95\%$ (core/tracker/*), $\geq 90\%$ (feature/*), $\geq 85\%$ (:app). Mutation kill rate $\geq 85\%$ (target $\geq 95\%$). All gaps tracked in docs/superpowers/specs/2026-04-30-sp3a-coverage-exemptions.md with per-line reason + reviewer + date.
5	Retroactive bluff audit of existing tests	Targeted in-flight audit (Task 1.0). Every Kotlin fake in core/testing/ consumed by SP-3a code gets the falsifiability protocol applied as a precondition to Phase 1. Existing EndpointConverterTest is re-rehearsed. Existing healthprobe contract test is re-rehearsed. Go-side audit defers to SP-3a-bridge. Broader sweep is Spec 2 Phase 6.
6	Mirror configuration source	Bundled mirrors.json $\cup$ user-supplied custom mirrors stored locally (Room). No remote update channel. New mirrors arrive via app updates and via user-supplied entries in :feature:tracker_settings UI.
7a	Cross-tracker fallback default UX	One-tap modal. Silent fallback rejected (violates Sixth Law clause 3); manual-only rejected (defeats SDK purpose).
7b	RuTor login policy	Anonymous by default. Login is invoked only when the SDK consumer calls a feature that requires it (e.g. CommentsTracker.addComment()).
7c	vasic-digital/Tracker-SDK mirroring policy	Originally: mirrored to all four upstreams (GitHub, GitFlic, GitLab, GitVerse). Revised 2026-04-30 mid-implementation per operator authorization: mirrored to GitHub + GitLab only. The Decoupled Reusable Architecture rule's recursive default is therefore relaxed for this submodule; if GitFlic/GitVerse access is later required, the submodule can be added to those upstreams without code changes.

1. Goals & Non-Goals

1.1 Goals (binding)

- **G1.** Define a tracker-agnostic SDK contract (TrackerClient + capability-based feature interfaces) that supports RuTracker today and RuTor as a proof of decoupling.
- **G2.** Decouple the existing RuTracker implementation behind the new contract with **zero behavioral change** verified by parity tests against the current Kotlin behavior.
- **G3.** Implement RuTor as a fully-decoupled tracker plugin, validating that the contract is real and not RuTracker-shaped in disguise.
- **G4.** Implement multi-mirror health tracking and fallback for both trackers, plus a one-tap cross-tracker fallback when all mirrors of the active tracker are exhausted.
- **G5.** Extract generic, reusable infrastructure (mirror manager, plugin registry, testing harness) into a new vasic-digital/Tracker-SDK submodule, mirrored to all four upstreams.
- **G6.** Add and enforce constitutional clauses 6.D (Behavioral Coverage Contract), 6.E (Capability Honesty), and 6.F (Anti-Bluff Submodule Inheritance). Cascade to all CLAUDE.md / AGENTS.md files at all levels.
- **G7.** Audit every Kotlin test fake consumed by SP-3a code via the falsifiability protocol; record evidence in .lava-ci-evidence/sp3a-bluff-audit/.
- **G8.** Land SP-3a as Android 1.2.0 with RuTor as a user-visible new feature; all eight Challenge Tests pass on a real Android device before tag.

1.2 Non-goals (explicitly deferred)

- **NG1.** HTTP/3 (QUIC) integration on the Android client — Spec 2 Phase 5 (depends on Cronet / OkHttp / size-budget call separately).
 - **NG2.** Brotli compression at the SDK layer beyond what vasic-digital/Middleware already provides — Spec 2.
 - **NG3.** Lazy-init framework, lock-free primitives audit, semaphore tuning — Spec 2.
 - **NG4.** Go-side lava-api-go/internal/rutacker/ refactor — moved to SP-3a-bridge, executes after SP-2 ships.
 - **NG5.** Removal of the Kotlin :proxy server — owned by SP-2.
 - **NG6.** Full mutation-test sweep of pre-existing Go test suite — Spec 2 Phase 6.
 - **NG7.** Remote mirror update channel — explicitly rejected per decision 6.
 - **NG8.** Public lava-app.tech endpoint reintroduction — already deleted per existing project memory; SP-3a does not change this.
-

2. Architecture Overview

2.1 Module map (after SP-3a)

Lava (this repo)	
├─ app/	(unchanged
module ID)	
├─ feature/	
│ └─ login/	(consumer
change: uses LavaTrackerSdk)	
│ └─ search_result/	(consumer
change)	
│ └─ topic/	(consumer
change)	
│ └─ forum/	(consumer
change)	
│ └─ favorites/	(consumer
change)	
│ └─ tracker_settings/	NEW – tracker
selection + custom mirrors UI	
├─ core/	
│ └─ tracker/	NEW
│ │ └─ api/	interfaces,
models, capability enum	
│ │ └─ client/	
LavaTrackerSdk facade + cross-tracker fallback policy	
│ │ └─ rutracker/	(renamed from
core/network/rutracker via git mv)	
│ │ └─ rutor/	NEW
│ │ └─ testing/	Lava-specific
test helpers (extends Tracker-SDK/testing)	
│ └─ network/	
│ │ └─ api/	(kept for
transitional backward compat)	
│ │ └─ impl/	
(SwitchingNetworkApi rewired to delegate to LavaTrackerSdk)	
│ │ └─ rutracker/	REMOVED – git
mv to core/tracker/rutracker	
│ └─ data/	(consumer
change in feature integrations)	
│ └─ domain/	(UseCases
retained but delegate to LavaTrackerSdk)	
│ └─ (other core/* unchanged)	
└─ submodules/	
└─ Tracker-SDK/	NEW – vasic-
digital/Tracker-SDK pinned hash	
└─ api/	Generic types:
MirrorUrl, Protocol, HealthState, RetryPolicy	
└─ mirror/	MirrorManager
engine + health checker	
└─ registry/	Generic
plugin-registry pattern	

	— testing/	Fake builders,
fixture	loader, falsifiability harness	
	— docs/	Submodule
README +	CONSTITUTION + AGENTS	
	— CLAUDE.md	
	— CONSTITUTION.md	

2.2 Dependency direction

```

:app
└─ feature/* (login, search_result, topic, forum, favorites,
tracker_settings)
    └─ :core:tracker:client      (LavaTrackerSdk facade)
        ├── :core:tracker:api    (interfaces + models)
        ├── :core:tracker:rutracker (impl)
        ├── :core:tracker:rutor   (impl)
        └─ submodules/tracker_sdk/ (mirror, registry,
testing, generic API)

```

`:core:tracker:rutracker` and `:core:tracker:rutor` both depend on `:core:tracker:api` and on `submodules/tracker_sdk/api/` (for `MirrorUrl`, `Protocol`, `HealthState`).

`:core:network:api` and `:core:network:impl` are **retained** during SP-3a as a transitional layer. After Phase 2 Task 2.6, `SwitchingNetworkApi` is rewired to be a thin adapter over `LavaTrackerSdk` — the existing `NetworkApi` interface keeps its 15-method shape so feature ViewModels keep compiling, but every call goes through the new SDK underneath. `:core:network:rutracker` is removed (its content moves to `:core:tracker:rutracker` via `git mv`, preserving history).

2.3 Build glue

A new convention plugin `lava.kotlin.tracker.module` extends `lava.kotlin.library` and pre-wires:

- `:core:tracker:api` dependency
- `submodules/tracker_sdk/api/`, `submodules/tracker_sdk/mirror/` dependencies
- Jsoup 1.15.3, OkHttp (matching root version), kotlinx-serialization
- Standard testing harness (`submodules/tracker_sdk/testing/`)

Tracker plugin modules (`:core:tracker:rutracker`, `:core:tracker:rutor`, plus any future tracker) apply this plugin instead of duplicating dependency declarations.

3. The vasic-digital/Tracker-SDK Submodule

3.1 Why a submodule, what's in scope

Per the Decoupled Reusable Architecture rule (root CLAUDE.md), code with a non-Lava-specific use case lives in a vasic-digital submodule. The pieces that pass that bar:

- **Mirror management & health-check engine.** Any multi-endpoint client (RSS aggregator, news scraper, forum client, tracker client) needs the same shape: a list of endpoints, per-endpoint health, fallback chain, periodic probing. Already overlaps with the existing vasic-digital/Recovery submodule's circuit-breaker — Tracker-SDK/mirror/ will *use* Recovery rather than re-implementing.
- **Generic plugin registry.** A ConcurrentHashMap of factories keyed by a string ID is reusable across any plugin-shaped system; the type parameters are what specialize it.
- **Testing harness.** DSL builders for test data, HTML/JSON fixture loading from classpath resources, the falsifiability-protocol rehearsal helper. None of this is tracker-shaped.

What is **explicitly out** of Tracker-SDK:

- TrackerClient, TrackerCapability, TrackerDescriptor, TorrentItem, SearchRequest, etc. — these are the tracker-domain contract and stay in :core:tracker:api. Once RuTor validates the shape (post-SP-3a), a future spec can decide whether to extract them.
- Anything that names a specific tracker (RuTracker, RuTor, Jackett, etc.).

3.2 Submodule constitution

submodules/tracker_sdk/CLAUDE.md and CONSTITUTION.md inherit from Lava's constitution and **add** a stricter rule: "no domain shape." Concretely, the submodule MUST NOT contain any class, function, file, or test resource that names a specific tracker, torrent site, or other Lava-domain entity. CI gate: `grep -rE '(?i)(rutracker|rutor|magnet|torrent|tracker\.org)'` submodules/tracker_sdk/ returns empty (with named exemption file for the word "tracker" appearing in generic contexts like "tracker-style client").

The submodule **inherits** Sixth Law clauses 6.A–6.F recursively; its own CLAUDE.md references the root and re-states the "no relaxation" clause.

3.3 Mirroring policy

Per decision 7c, vasic-digital/Tracker-SDK is mirrored to GitHub + GitFlic + GitLab + GitVerse from initial creation. `scripts/sync-mirrors.sh` (which Lava already has for the main repo) gets a parallel invocation for the submodule. Branch protection rules on each upstream require the locally-run CI evidence file before merging to the default branch — same gate Lava uses.

3.4 Versioning

Tracker-SDK starts at `0.1.0` on creation. SP-3a pins to whatever hash the submodule has at the moment SP-3a Phase 1 completes. The pin is **frozen by default**; updating it is a deliberate PR with a documented reason.

3.5 Initial public surface

```
package lava.sdk.api                                (Tracker-SDK/api/)
    data class MirrorUrl(val url: String, val isPrimary: Boolean =
false, val priority: Int = 0,
                        val protocol: Protocol = Protocol.HTTPS,
val region: String? = null)
    enum class Protocol { HTTP, HTTPS, HTTP3 }
    enum class HealthState { HEALTHY, DEGRADED, UNHEALTHY, UNKNOWN }
    data class MirrorState(val mirror: MirrorUrl, val health:
HealthState,
                        val lastCheck: Instant?, val
consecutiveFailures: Int)
    data class FallbackPolicy(val maxAttempts: Int = 3, val
perAttemptTimeout: Duration = 10.seconds,
                        val degradeAfter: Int = 1, val
unhealthyAfter: Int = 3)
    class MirrorUnavailableException(val tried: List<MirrorUrl>) :
Exception()

package lava.sdk.mirror                            (Tracker-SDK/mirror/)
    interface MirrorManager {
        suspend fun getHealthyMirror(endpointGroupId: String):
MirrorUrl?
        suspend fun <T> executeWithFallback(endpointGroupId: String,
op: suspend (MirrorUrl) -> T): T
        fun observeHealth(endpointGroupId: String):
Flow<List<MirrorState>>
        suspend fun probeAll(endpointGroupId: String)
        suspend fun reportSuccess(endpoint: MirrorUrl)
        suspend fun reportFailure(endpoint: MirrorUrl, cause:
Throwable)
    }
    class DefaultMirrorManager(...) : MirrorManager
    interface HealthProbe {
```

```

suspend fun probe(endpoint: MirrorUrl): HealthState
}

package lava.sdk.registry                (Tracker-SDK/registry/)
interface PluginRegistry<D, P> {
    fun register(descriptor: D, factory: PluginFactory<D, P>)
    fun unregister(id: String)
    fun get(id: String, config: PluginConfig): P
    fun list(): List<D>
}
interface PluginFactory<D, P> {
    val descriptor: D
    fun create(config: PluginConfig): P
}
class DefaultPluginRegistry<D : HasId, P>(...) :
PluginRegistry<D, P>

package lava.sdk.testing                (Tracker-SDK/testing/)
class FakeMirrorManager : MirrorManager (configurable health
states)
class HtmlFixtureLoader(val resourcePath: String) (load from
classpath, freshness check)
inline fun <reified T> falsifiabilityRehearsal(
    name: String, mutator: () -> AutoCloseable,
expectFailureMessage: Regex,
    runner: () -> Unit
): RehearsalRecord

```

The falsifiabilityRehearsal helper is the load-bearing testing primitive. It runs:

1. Apply mutator to deliberately break the system under test.
2. Run runner.
3. Assert that runner throws or fails with a message matching expectFailureMessage.
4. Revert the mutator (via AutoCloseable.close()).
5. Re-run runner, assert it passes.
6. Return a RehearsalRecord(testName, mutationDescription, observedMessage, revertedSuccessfully) for evidence-pack capture.

This is what Sixth Law clause 6.A requires for every contract test, and what we'll cascade as the standard pattern.

4. Core Interfaces & Data Model

(:core:tracker:api)

4.1 The TrackerClient contract

```
// :core:tracker:api/src/main/kotlin/lava/tracker/api/
// TrackerClient.kt
interface TrackerClient : AutoCloseable {
    val descriptor: TrackerDescriptor
    suspend fun healthCheck(): Boolean
    fun <T : TrackerFeature> getFeature(featureClass: KClass<T>): T?
}

interface TrackerFeature // marker interface; feature interfaces
// extend this
```

getFeature returns null when the tracker does not support the requested feature. This **eliminates** the "Not implemented" stub pattern that exists today (e.g., ProxyNetworkApi.checkAuthorized() and ProxyNetworkApi.download() return false / throw despite being declared in the interface). Capability declared ⇒ feature interface returned ⇒ method works. This is what Constitutional clause 6.E (Capability Honesty) will enforce.

4.2 Feature interfaces

```
interface SearchableTracker : TrackerFeature {
    suspend fun search(request: SearchRequest, page: Int = 0):
        SearchResult
}

interface BrowsableTracker : TrackerFeature {
    suspend fun browse(category: String?, page: Int): BrowseResult
    suspend fun getForumTree():
        ForumTree? // null when tracker has no forum (RuTor)
}

interface TopicTracker : TrackerFeature {
    suspend fun getTopic(id: String): TopicDetail
    suspend fun getTopicPage(id: String, page: Int): TopicPage
}

interface CommentsTracker : TrackerFeature {
    suspend fun getComments(topicId: String, page: Int):
        CommentsPage
    suspend fun addComment(topicId: String, message: String):
        Boolean
}

interface FavoritesTracker : TrackerFeature {
```

```

suspend fun list(): List<TorrentItem>
suspend fun add(id: String): Boolean
suspend fun remove(id: String): Boolean
}

interface AuthenticatableTracker : TrackerFeature {
    suspend fun login(req: LoginRequest): LoginResult
    suspend fun logout()
    suspend fun checkAuth(): AuthState
}

interface DownloadableTracker : TrackerFeature {
    suspend fun downloadTorrentFile(id: String): ByteArray
    fun getMagnetLink(id: String): String?
}

```

Capability enum (used both for descriptor declarations and for runtime queries):

```

enum class TrackerCapability {
    SEARCH, BROWSE, FORUM, TOPIC, COMMENTS, FAVORITES,
    TORRENT_DOWNLOAD, MAGNET_LINK, AUTH_REQUIRED,
    CAPTCHA_LOGIN, RSS, UPLOAD, USER_PROFILE
}

```

4.3 Tracker descriptor

```

data class TrackerDescriptor(
    val trackerId: String,           // "rutracker", "rutor"
    val displayName: String,        // "RuTracker.org",
    // "RuTor.info"
    val baseUrls: List<MirrorUrl>,   // primary + mirrors
    val capabilities: Set<TrackerCapability>,
    val authType: AuthType,         // NONE, FORM_LOGIN,
    // CAPTCHA_LOGIN, OAUTH, API_KEY
    val encoding: String,           // "UTF-8",
    // "Windows-1251"
    val expectedHealthMarker: String // string content
    // present on the tracker's root page;
    // health probe asserts
    // response body contains this
)

enum class AuthType { NONE, FORM_LOGIN, CAPTCHA_LOGIN, OAUTH,
    API_KEY }

```

expectedHealthMarker is the load-bearing field for the "200 OK but the page is a Russian government block notice" failure mode — without it, healthCheck() would falsely report HEALTHY for a captive-portal redirect. RuTracker's marker is "rutracker" (case-insensitive substring); RuTor's is "RuTor".

4.4 Common data model

Tracker-agnostic types replacing the existing RuTracker-specific DTOs:

```
data class TorrentItem(
    val trackerId: String,
    val torrentId: String,
    val title: String,
    val sizeBytes: Long?,
    val seeders: Int?,
    val leechers: Int?,
    val infoHash: String?,
    val magnetUri: String?,
    val downloadUrl: String?,
    val detailUrl: String?,
    val category: String?,
    val publishDate: Instant?,
    val metadata: Map<String, String> = emptyMap() // tracker-
        specific extras (e.g. "comments_count")
)

data class SearchRequest(
    val query: String,
    val categories: List<String> = emptyList(),
    val sort: SortField = SortField.DATE,
    val sortOrder: SortOrder = SortOrder.DECENDING,
    val author: String? = null,
    val period: TimePeriod? = null
)

enum class SortField { DATE, SEEDERS, LEECHERS, SIZE, RELEVANCE,
    TITLE }
enum class SortOrder { ASCENDING, DECENDING }
enum class TimePeriod { LAST_DAY, LAST_WEEK, LAST_MONTH,
    LAST_YEAR, ALL_TIME }

data class SearchResult(val items: List<TorrentItem>, val
    totalPages: Int, val currentPage: Int)
data class BrowseResult(val items: List<TorrentItem>, val
    totalPages: Int, val currentPage: Int,
    val category: ForumCategory?)
data class TopicDetail(val torrent: TorrentItem, val description:
    String?, val files: List<TorrentFile>)
data class TopicPage(val topic: TopicDetail, val totalPages: Int,
    val currentPage: Int)
data class TorrentFile(val name: String, val sizeBytes: Long?)
data class CommentsPage(val items: List<Comment>, val totalPages:
    Int, val currentPage: Int)
data class Comment(val author: String, val timestamp: Instant?,
    val body: String)
data class ForumTree(val rootCategories: List<ForumCategory>)
data class ForumCategory(val id: String, val name: String, val
    parentId: String?, val children: List<ForumCategory>)
data class LoginRequest(val username: String, val password:
    String, val captcha: CaptchaSolution? = null)
```

```

data class LoginResult(val state: AuthState, val sessionToken:
    String? = null,
                        val captchaChallenge: CaptchaChallenge? =
                            null)
sealed class AuthState {
    object Authenticated : AuthState()
    object Unauthenticated : AuthState()
    data class CaptchaRequired(val challenge: CaptchaChallenge) :
        AuthState()
}
data class CaptchaChallenge(val sid: String, val code: String,
    val imageUrl: String)
data class CaptchaSolution(val sid: String, val code: String, val
    value: String)

```

4.5 DTO model mappers

In `:core:tracker:rutracker/mapper/`:

- `ForumDtoMapper : ForumDto → ForumTree`
- `SearchPageMapper : SearchPageDto → SearchResult`
- `TopicMapper : TopicPageDto → TopicDetail + TopicPage`
- `CommentsMapper : CommentsPageDto → CommentsPage`
- `TorrentMapper : TorrentDto → TorrentItem`
- `AuthMapper : AuthResponseDto → LoginResult / AuthState`
- `FavoritesMapper : FavoritesDto → List<TorrentItem>`

RuTracker-specific extras (e.g., post-element types from `PostElementDto`'s 18 variants) go into metadata keyed by namespaced strings like `"rutracker.post.element_type"`. UI consumers either read the namespaced key (with explicit awareness they're consuming RuTracker-specific data) or they ignore metadata.

4.6 Type-safe consumer pattern

```

val tracker: TrackerClient = lavaTrackerSdk.getActiveTracker()
val search = tracker.getFeature(SearchableTracker::class)
?: throw IllegalStateException("Tracker $
    {tracker.descriptor.trackerId} does not advertise SEARCH
    capability")
val results = search.search(SearchRequest(query = "ubuntu 24.04"))

```

ViewModel-level pattern (using the `LavaTrackerSdk` facade so cross-tracker fallback is available):

```

viewModelScope.launch {
    intent {
        reduce { state.copy(loading = true) }
        val outcome = lavaTrackerSdk.search(SearchRequest(query =
            state.query))
    }
}

```

```

when (outcome) {
  is SearchOutcome.Success -> reduce { state.copy(items =
    outcome.result.items, loading = false) }
  is SearchOutcome.CrossTrackerFallbackProposed ->
    postSideEffect(
      SearchSideEffect.OfferAlternateTracker(outcome.failedTrackerId,
        outcome.proposedTrackerId)
    )
  is SearchOutcome.Failure -> reduce { state.copy(error =
    outcome.cause.message, loading = false) }
}
}
}

```

5. Mirror Management & Cross-Tracker Fallback

5.1 Per-tracker mirror health

MirrorManager (from Tracker-SDK/mirror/) tracks state per tracker:

- **HEALTHY** — last probe within the last 15min succeeded (status 200, body contains expectedHealthMarker, response time <5s).
- **DEGRADED** — most recent probe failed but the previous two probes succeeded, OR most recent probe succeeded with response time 5–10s, OR most recent probe returned status 5xx but response was received.
- **UNHEALTHY** — ≥3 consecutive probe failures, OR ≥2 consecutive probes that timed out (>10s no response).
- **UNKNOWN** — never probed (state at first app launch before the periodic worker has run).

State is persisted to Room (tracker_mirror_health table: tracker_id, mirror_url, state, last_check_at, consecutive_failures). Persistence guarantees that health survives app restarts — the app does not start in UNKNOWN state for mirrors that were known-bad on previous run.

5.2 Health probe scheduling

WorkManager periodic worker MirrorHealthCheckWorker runs every 15 minutes (battery-aware constraint: only on connected network, not requiring charging). Worker invokes MirrorManager.probeAll() for every registered tracker.

Each probe is:

1. GET / against the mirror with 5s timeout.

2. Assert HTTP 200 (or 301/302 redirecting to same domain).
3. Assert response body contains descriptor.expectedHealthMarker (case-insensitive substring).
4. Record responseTimeMs.
5. Update MirrorState accordingly.

5.3 Fallback chain executor

```
suspend fun <T> executeWithFallback(
    endpointGroupId: String,
    op: suspend (MirrorUrl) -> T
): T {
    val states = stateFor(endpointGroupId).sortedWith(
        compareBy(
            { healthRank(it.health) },      // HEALTHY=0, DEGRADED=1,
            UNKNOWN=2, UNHEALTHY=3
            { it.mirror.priority }
        )
    )
    val tried = mutableListOf<MirrorUrl>()
    for (state in states) {
        if (state.health == HealthState.UNHEALTHY) continue
        tried += state.mirror
        try {
            val result = withTimeout(policy.perAttemptTimeout) {
                op(state.mirror)
            }
            reportSuccess(state.mirror)
            return result
        } catch (t: Throwable) {
            reportFailure(state.mirror, t)
            // continue to next mirror
        }
    }
    throw MirrorUnavailableException(tried)
}
```

5.4 Cross-tracker fallback (Lava-specific, in :core:tracker:client)

When `MirrorManager.executeWithFallback` throws `MirrorUnavailableException` AND the operation belongs to a feature interface supported by an alternate registered tracker, `LavaTrackerSdk` does **not** auto-retry. Instead it returns a typed outcome:

```
sealed class SearchOutcome {
    data class Success(val result: SearchResult, val viaTracker: String) : SearchOutcome()
    data class CrossTrackerFallbackProposed(
        val failedTrackerId: String,
        val proposedTrackerId: String,
```

```

    val capability: TrackerCapability,
    val resumeWith: suspend () -> SearchOutcome
) : SearchOutcome()
data class Failure(val cause: Throwable, val triedTrackers:
    List<String>) : SearchOutcome()
}

```

The feature ViewModel receives CrossTrackerFallbackProposed and emits a side effect that the UI renders as a one-tap modal:

RuTracker is unavailable. All known mirrors are unreachable. Try the same search on **RuTor**?

[Try RuTor] [Cancel]

User taps "Try RuTor" → ViewModel invokes outcome.resumeWith() → operation retries on the proposed tracker. User taps "Cancel" → original Failure outcome propagates. The user is **always** notified of the tracker switch via a transient banner ("Searching RuTor — RuTracker mirrors unreachable") for the duration of the fallback session.

Why one-tap rather than silent: decision 7a-ii. Silent fallback violates Sixth Law clause 3 (primary user-visible signal must reflect what actually happened) — a search result labeled as RuTracker that's actually from RuTor is a category-bluff.

5.5 Mirror configuration

Bundled JSON shipped with the app at :app/src/main/assets/mirrors.json:

```

{
  "version": 1,
  "trackers": {
    "rutracker": {
      "expectedHealthMarker": "rutracker",
      "mirrors": [
        {"url": "https://rutracker.org", "isPrimary": true,
          "priority": 0, "protocol": "HTTPS"},
        {"url": "https://rutracker.net",
          "priority": 1, "protocol": "HTTPS"},
        {"url": "https://rutracker.cr",
          "priority": 2, "protocol": "HTTPS"}
      ]
    },
    "rutor": {
      "expectedHealthMarker": "RuTor",
      "mirrors": [
        {"url": "https://rutor.info", "isPrimary": true,
          "priority": 0, "protocol": "HTTPS"},

```

```

        {"url": "https://rutor.is",
         "priority": 1, "protocol": "HTTPS"},
        {"url": "https://www.rutor.info",
         "priority": 2, "protocol": "HTTPS"},
        {"url": "https://www.rutor.is",
         "priority": 3, "protocol": "HTTPS"},
        {"url": "http://6tor.org",
         "priority": 4, "protocol": "HTTP", "region": "ipv6-only"}
    ]
}
}
}
}

```

User-supplied mirrors are stored in Room (tracker_mirror_user, columns: tracker_id,url,priority,protocol,added_at). User-supplied entries can only **add** mirrors or **change priority**; they cannot delete bundled mirrors. To deprioritize a bundled mirror to "never use," user sets a user-mirror entry for the same URL with priority=Int.MAX_VALUE. UI in :feature:tracker_settings exposes both lists.

5.6 Falsifiability rehearsal for fallback

Mandatory test in :core:tracker:client:

```

@Test
fun fallbackChainExecutorRehearsal() {
    val rehearsal = falsifiabilityRehearsal(
        name = "fallback chain skips unhealthy mirror",
        mutator = { manager.markHealthy(mirror0) }, // deliberately mark broken mirror as healthy
        expectFailureMessage = Regex("""attempted broken mirror.*"""),
        runner = { runBlocking { manager.executeWithFallback("rutor")
            { url -> client.fetch(url) } } }
    )
    evidence.record(rehearsal)
}

```

Records to .lava-ci-evidence/sp3a-bluff-audit/fallback-rehearsal-<commit-sha>.json.

6. RuTor Implementation

6.1 Module: :core:tracker:rutor

```

core/tracker/rutor/
├─ build.gradle.kts                (applies
lava.kotlin.tracker.module)
├─ src/
│   └─ main/kotlin/lava/tracker/rutor/

```

```

|   |— RuTorClient.kt           implements TrackerClient +
applicable feature interfaces
|   |— RuTorDescriptor.kt
|   |— RuTorClientFactory.kt
|   |— http/
|   |   |— RuTorHttpClient.kt    internal HTTP client (OkHttp-
based)
|   |   |— parser/
|   |       |— RuTorSearchParser.kt
|   |       |— RuTorBrowseParser.kt
|   |       |— RuTorTopicParser.kt
|   |       |— RuTorLoginParser.kt
|   |       |— RuTorCommentsParser.kt
|   |       |— RuTorDateParser.kt    Russian month abbreviations +
Сегодня/Вчера
|   |   |— test/
|   |       |— kotlin/...          unit tests + parser tests
|   |       |— resources/fixtures/rutor/
|   |           |— search/
|   |               {normal,empty,edge_columns,cyrillic,malformed}.html
|   |               |— browse/{...}.html
|   |               |— topic/{...}.html
|   |               |— login/{success,failure}.html

```

6.2 RuTor descriptor

```

object RuTorDescriptor : TrackerDescriptor {
    override val trackerId = "rutor"
    override val displayName = "RuTor.info"
    override val baseUrls = listOf(/* loaded from mirrors.json at
runtime */)
    override val capabilities = setOf(
        SEARCH, BROWSE, TOPIC, COMMENTS, TORRENT_DOWNLOAD,
        MAGNET_LINK, RSS, AUTH_REQUIRED
    )
    // Note: FORUM and FAVORITES are NOT in the set. RuTor has no
forum tree
    // and no list-favorites endpoint comparable to RuTracker.
    override val authType = AuthType.FORM_LOGIN // simple form, no
captcha
    override val encoding = "UTF-8"
    override val expectedHealthMarker = "RuTor"
}

```

6.3 URL patterns

Search: /search/{page}/{cat}/{method}{scope}0/{sort}/{query}
where: page is 0-indexed,
 cat is numeric category ID 0..17 (0 = all),
 method ∈ { 0 (any words), 1 (all words), 2
(exact phrase), 3 (logical) },
 scope ∈ { 0 (title+desc), 1 (title only) },

```

                                sort ∈ 0..11 (date asc/desc, seeders asc/
desc, ...)
Browse:      /browse/{page}/{cat}/{method}/{sort}
Topic:       /torrent/{id}/{slug}
Download:    /download/{id}      (302 → d.rutor.info/download/{id})
Login:       POST /users.php?login form: nick={user}
&password={pass}
Comments:    embedded in topic page, no separate URL
RSS:         /rss.php?category={id}

```

6.4 HTML parsing — variable column count

The hazard from PDF Section 1.5 documented in concrete form: search result rows have a comments-count `<td>` that is **sometimes present, sometimes absent**, shifting all subsequent columns by 1. Positional `td:nth-of-type(N)` selectors fail unpredictably.

Solution: **content-based selectors** borrowed from Jackett's `rutor.yml` definition (battle-tested in production, MIT-licensed):

```

// :core:tracker:rutor/src/main/kotlin/lava/tracker/rutor/parser/
// RuTorSearchParser.kt
class RuTorSearchParser {
    fun parse(html: String): SearchResult {
        val doc = Jsoup.parse(html)
        val rows = doc.select("tr:has(td:has(a[href^=magnet:?xt=]))")
        val items = rows.map { row -> parseRow(row) }
        val totalPages = doc.select("td.pages a").mapNotNull {
            it.text().toIntOrNull() }.maxOrNull() ?: 1
        // currentPage extracted from URL or ".pages b" element
        return SearchResult(items, totalPages, currentPage =
            parseCurrentPage(doc))
    }

    private fun parseRow(row: Element): TorrentItem {
        val titleLink = row.selectFirst("td:nth-of-type(2) a[href^=/
torrent/]")
        ?: error("RuTor row missing title link: $row")
        val magnet = row.selectFirst("a[href^=magnet:?
xt=]")?.attr("href")
        val downloadHref = row.selectFirst("a.downgif")?.attr("href")
        // Content-based size selector – finds the <td> regardless of
        // column position
        val sizeText = row.select("td").firstOrNull { td ->
            td.text().matches(Regex("""\d+(\.\d+)? (GB|MB|KB|B|TB)"""))
        }?.text()
        val seeders = row.selectFirst("td
span.green")?.text()?.toIntOrNull()
        val leechers = row.selectFirst("td
span.red")?.text()?.toIntOrNull()
        val date = row.selectFirst("td:nth-of-
type(1)")?.text()?.let(RuTorDateParser::parse)

```

```

val infoHash = magnet?.let { Regex("""[A-Fa-f0-9]{40}""").find(it)?.value }
return TorrentItem(
    trackerId = "rutor",
    torrentId = extractIdFromHref(titleLink.attr("href")),
    title = titleLink.text(),
    sizeBytes = sizeText?.let(::parseSizeToBytes),
    seeders = seeders, leechers = leechers,
    infoHash = infoHash, magnetUri = magnet,
    downloadUrl = downloadHref?.let {
        resolveAgainst(currentMirror, it) },
    detailUrl = resolveAgainst(currentMirror,
        titleLink.attr("href")),
    category = null, // category not present in search rows;
        available on detail page
    publishDate = date,
    metadata = emptyMap()
)
}
}
}

```

6.5 Russian date parsing

```

// :core:tracker:rutor/src/main/kotlin/lava/tracker/rutor/parser/
// RuTorDateParser.kt
object RuTorDateParser {
    private val months = mapOf(
        "Янв" to 1, "Фев" to 2, "Мар" to 3, "Апр" to 4, "Май" to 5,
        "Июн" to 6,
        "Июл" to 7, "Авг" to 8, "Сен" to 9, "Окт" to 10, "Ноя" to 11,
        "Дек" to 12
    )
    fun parse(s: String, now: () -> Instant = Instant::now):
        Instant? {
        val trimmed = s.trim()
        return when {
            trimmed.equals("Сегодня", ignoreCase = true) ->
                now().truncatedTo(ChronoUnit.DAYS)
            trimmed.equals("Вчера", ignoreCase = true) ->
                now().truncatedTo(ChronoUnit.DAYS).minus(1,
                    ChronoUnit.DAYS)
            else -> {
                val match = Regex("""(\d{1,2})\s+(\S+)\s+
                    (\d{2,4})""").find(trimmed) ?: return null
                val (day, monthAbbr, year) = match.destructured
                val month = months[monthAbbr.take(3)] ?: return null
                val fullYear = if (year.length == 2) 2000 + year.toInt()
                    else year.toInt()
                LocalDate.of(fullYear, month,
                    day.toInt()).atStartOfDay(ZoneOffset.UTC).toInstant()
            }
        }
    }
}
}
}

```

6.6 Login flow

```
class RuTorAuthenticator(private val http: RuTorHttpClient) :
    AuthenticatableTracker {
    override suspend fun login(req: LoginRequest): LoginResult {
        val response = http.postForm("/users.php?login", mapOf(
            "nick" to req.username,
            "password" to req.password,
            "login" to "Bxoq"
        ))
        val cookieHeader = response.headers["Set-Cookie"].orEmpty()
        val userid = Regex("""userid=([^\;]+)""").find(cookieHeader.joinToString())?.groupValues?.get(1)
        return if (userid != null) {
            LoginResult(state = AuthState.Authenticated, sessionToken =
                userid)
        } else {
            LoginResult(state = AuthState.Unauthenticated)
        }
    }
    override suspend fun logout() { http.clearCookies() }
    override suspend fun checkAuth(): AuthState =
        if (http.hasCookie("userid")) AuthState.Authenticated else
            AuthState.Unauthenticated
}
```

Per decision 7b-ii (anonymous-by-default),
RuTorClient.getFeature(AuthenticatableTracker::class) returns this implementation but LavaTrackerSdk does not call login() at startup. Login is triggered only when an authenticated operation (currently addComment()) is attempted; if checkAuth() returns Unauthenticated and credentials are configured, login is invoked transparently before retrying the operation.

6.7 Test fixtures (binding)

Per the testing-strategy contract, ≥ 5 fixtures per parser:

Parser	Fixtures (minimum)
RuTorSearchParser	normal.html (multiple results, all columns), empty.html (zero results), edge_columns.html (comments-count column missing on some rows), cyrillic.html (Cyrillic titles + special chars), malformed.html (truncated/error page)
RuTorBrowseParser	normal, empty, deep_pagination, single_result, malformed
RuTorTopicParser	normal, with_files, with_long_description, no_magnet, malformed
RuTorLoginParser	

Parser	Fixtures (minimum)
	success_with_userid_cookie, failure_wrong_password, failure_account_locked, redirect_to_home, malformed

All fixtures are dated in their filename (search-normal-2026-04-30.html) and tracked by scripts/check-fixture-freshness.sh (warns when >30 days old, blocks tag when >60 days old).

7. Mappers, Adapter, and Backward Compatibility

7.1 Phase-2 backward compat: SwitchingNetworkApi rewire

The existing `:core:network:impl/SwitchingNetworkApi` currently routes `NetworkApi` calls to one of two concrete `NetworkApi` impls (the proxy variant or the direct-RuTracker variant). After Phase 2 Task 2.6, `SwitchingNetworkApi` is **rewired** to delegate to `LavaTrackerSdk` underneath:

```
class SwitchingNetworkApi @Inject constructor(
    private val sdk: LavaTrackerSdk,
    private val ruTrackerMappers: RuTrackerDtoMappers // reverse
                mappers: new model → legacy DTO
) : NetworkApi {
    override suspend fun getSearchPage(token: String?, query:
        String, ...): SearchPageDto {
        val result = sdk.search(SearchRequest(query, ...))
        return ruTrackerMappers.searchResultToDto(result)
    }
    // ... 14 more methods, each translating between old DTO and new
    // model via mappers.
    // The full reverse-mapper API surface is enumerated in the
    // implementation plan,
    // not in this design doc – see plan $RuTrackerDtoMappers.
}
```

This is the load-bearing backward-compat hook. Feature ViewModels keep calling `NetworkApi`; the wire underneath flows through `LavaTrackerSdk`. After SP-3a-bridge ships and the Go side is also on the new contract, Spec 2 retires the `NetworkApi` interface entirely.

7.2 Endpoint sealed type (unchanged)

The existing Endpoint sealed type — currently `Endpoint.LAN` and `Endpoint.Direct` (per project memory, `Endpoint.Proxy` was removed) — is **not** modified by SP-3a. Tracker selection (which tracker to query) is orthogonal to endpoint selection (how the app reaches the api-go server). The `:feature:tracker_settings` UI is a sibling to the existing endpoint-settings UI, not a replacement.

7.3 Permissive LAN TLS (unchanged)

Per project memory, the Android client must connect to LAN api-go over HTTPS without manual cert install (permissive trust scoped to LAN endpoints only). SP-3a does **not** modify this behavior — tracker connections (`rutracker.org` / `rutor.info`) use standard system trust because they're internet-facing TLS, while LAN connections to api-go retain the existing permissive trust. The two trust policies live in separate OkHttp client configurations and SP-3a does not merge them.

8. Testing Strategy & Anti-Bluff Contract

8.1 Coverage contract (decision 4-C)

Behavioral coverage (binding). Every public method of every interface added in Phase 1 has at least one **real-stack test** that traverses: `feature` → `:core:tracker:client` (`LavaTrackerSdk`) → `:core:tracker:registry` → `TrackerClient` impl → `:core:tracker:mirror` (`MirrorManager`) → `fixture HTTP` layer → `fixture HTML` → asserted user-visible state. The fixture HTTP layer is the **only** boundary fake permitted in the chain.

Line coverage targets (secondary, gated):

- `:core:tracker:api`, `:core:tracker:client`, `:core:tracker:rutracker`, `:core:tracker:ru` ≥95%
- `submodules/tracker_sdk/**`: ≥95%
- `feature/tracker_settings`, other modified `feature/*`: ≥90%
- `:app`: ≥85% (lower because Compose Activity glue is hard to unit-test)

Mutation kill rate (gated): ≥85% minimum, ≥95% target, on `:core:tracker:*` and `submodules/tracker_sdk/**`. PITest on Kotlin; existing `scripts/mutation.sh` for Go (deferred to SP-3a-bridge).

Exemption ledger: docs/superpowers/specs/2026-04-30-sp3a-coverage-exemptions.md — every uncovered line gets an entry: path:line – reason – reviewer – date. Exemptions are reviewed in PR. Blanket waivers are forbidden.

8.2 Task 1.0 — In-flight bluff audit (precondition to Phase 1)

For each of the following test fakes consumed by SP-3a, run the falsifiability protocol and record evidence:

Fake	Real counterpart	Mutation to a
TestEndpointsRepository	EndpointsRepositoryImpl	Remove duplicate rejection logic assert that test using the fake when adding a duplicate endpoint
TestBookmarksRepository	BookmarksRepositoryImpl	Disable Room constraint enforcement in fake → assert test asserting "duplicate add fails" actually fails
TestAuthService	AuthServiceImpl	Drop login-token storage → assert that test relying persisted auth state fails
TestLocalNetworkDiscoveryService	LocalNetworkDiscoveryServiceImpl	Drop _lava._tcp.local suffix handling assert that test parsing real-for service names
Existing EndpointConverterTest	(the test itself)	Mutate the converter to drop known endpoint variant → assert test fails
Existing healthprobe contract test	(the test itself)	Re-introduce the historical --health flag bug in

Fake	Real counterpart	Mutation to a
New FakeTrackerClient	RuTrackerClient, RuTorClient	compose file — assert test fails Make fake return success when descriptor declared no SEARCH capability → as capability-honored test fails

Evidence written to `.lava-ci-evidence/sp3a-bluff-audit/<timestamp>-<test-name>.json` with the Test/Mutation/Observed/Reverted quartet from clause 6.A.

8.3 Per-feature Challenge Tests (binding before tag)

Eight Challenge Tests, each runnable as a Compose UI test on a real Android device:

1. **C1.** App launches → tracker selection screen → user picks RuTracker → home displays.
2. **C2.** Authenticated search on RuTracker returns ≥ 1 result with parseable size and seeders.
3. **C3.** Search on RuTor (anonymous) returns ≥ 1 result with parseable size and seeders.
4. **C4.** Switching active tracker from RuTracker to RuTor re-runs the same query and shows different results.
5. **C5.** View topic detail → torrent file list renders → magnet URI present.
6. **C6.** Download `.torrent` file → file written to expected path → file is a valid bencoded torrent.
7. **C7.** Cross-tracker fallback modal: simulate all RuTracker mirrors UNHEALTHY, perform search, modal appears, tap "Try RuTor", results from RuTor render with the "results from RuTor" banner.
8. **C8.** Cross-tracker fallback modal cancel: same setup as C7, tap "Cancel", explicit failure UI renders (no silent fallback).

Each Challenge Test PR records its falsifiability rehearsal: deliberate break (e.g. for C2: throw inside the search use case), observe Challenge Test failure with specific message, revert, observe pass. Recorded in `.lava-ci-evidence/sp3a-challenges/<C-id>-<commit-sha>.json`.

8.4 Fixture freshness

`scripts/check-fixture-freshness.sh` runs as a pre-push hook step. Fixtures older than 30 days emit a WARN; fixtures older than 60 days BLOCK any tag operation. Re-scraping is documented in `docs/refactoring/decoupling/refresh-fixtures.md` (to be created in Phase 7).

8.5 Real-tracker integration tests

Tagged with `@RealTracker` annotation, excluded from default test runs, executed only by `scripts/ci.sh --real-trackers`:

- Login to RuTracker with real credentials (`<redacted-per-$6.H> / <redacted-per-$6.H>`); assert non-null session cookie.
- Login to RuTor with same credentials; assert authenticated state via `userid` cookie.
- Real search on each tracker: `query="ubuntu"`, expect ≥ 3 results, assert all results have non-empty `title` and parseable `infoHash` or `magnetUri`.
- Real download of a `.torrent` from each tracker; assert file is a valid bencoded torrent (parseable `info.pieces` field).
- Real mirror health probe for each mirror in `mirrors.json`; record final `HealthState`; warn if any HEALTHY-marked primary mirror probes UNHEALTHY (signal that `mirrors.json` is drifting).

These tests require network access and valid credentials; they run as a separate "smoke test" suite invoked manually before each release tag, output captured in `.lava-ci-evidence/sp3a-real-tracker-smoke/<tag>.json`.

8.6 Local-only CI gate

`scripts/ci.sh` (new in this spec, would have been SP-2; pulled forward) is the single entry point. Subset run by pre-push hook based on changed files:

Changed path	ci.sh subset run
<code>core/tracker/api/**</code>	unit tests, line-coverage, mutation, behavioral-coverage report
<code>core/tracker/{rutracker,rutor}/**</code>	parser tests + fixtures, behavioral-coverage
<code>core/tracker/client/**</code>	fallback rehearsal, cross-tracker rehearsal, unit
<code>core/tracker/mirror/**</code> (Tracker-SDK)	mirror falsifiability rehearsal, health-probe unit
<code>feature/**</code>	Compose UI tests subset, behavioral-coverage

Changed path	ci.sh subset run
app/**	full Challenge Test suite (slow; runs in background)
Constitutional docs	constitution-doc parser check
Anywhere	spotless, hosted-CI-file forbidden-paths check, host-power forbidden-command regex

Pre-push hook is **not bypassable** in routine work. `--no-verify` is reserved for documented emergencies and any such use must be noted in the next commit message body (per existing constitutional rule).

8.7 Tag gate

`scripts/tag.sh` refuses to operate without:

1. `.lava-ci-evidence/<tag>/ci.sh.json` showing full `ci.sh` run on the exact tagged commit.
2. `.lava-ci-evidence/<tag>/challenges/` showing all 8 Challenge Tests pass on a real Android device, with screenshots attached.
3. `.lava-ci-evidence/<tag>/bluff-audit/` showing falsifiability rehearsals for every fake touched.
4. `.lava-ci-evidence/<tag>/mirror-smoke/` showing real-tracker smoke tests passed within last 24 hours.
5. Per-mirror SHA verification across all four upstreams (per existing 6.C clause).

9. Constitutional Updates

9.1 Root CLAUDE.md — three new clauses

6.D — Behavioral Coverage Contract. Adopted body:

Coverage is measured behaviorally, not lexically. Every public method of every interface added under `core/tracker/api/`, `submodules/tracker_sdk/api/`, or any future SDK contract module **MUST** have at least one real-stack test that traverses the same code path a user's action triggers. Line coverage is reported as a secondary metric. Uncovered lines after the behavioral pass are exempted only via an entry in the per-spec exemption ledger (`docs/superpowers/specs/<spec>-coverage-exemptions.md`) naming the line, the reason, the reviewer, and the date. Blanket coverage waivers are forbidden.

6.E — Capability Honesty. Adopted body:

A TrackerDescriptor (or any future descriptor of a feature-bearing component) that declares a capability MUST cause `getFeature()` to return a non-null implementation for the corresponding feature interface. The historical "Not implemented" stub pattern (e.g., `ProxyNetworkApi.checkAuthorized()` returning `false` despite being declared) is a constitutional violation. Capability declared $\Rightarrow$ feature interface returned $\Rightarrow$ at least one real-stack test exists for the capability. CI gate: a unit test enumerates every descriptor, every declared capability, and asserts the corresponding `getFeature()` call returns non-null.

6.F — Anti-Bluff Submodule Inheritance. Adopted body:

Clauses 6.A through 6.E inherit recursively to every vasic-digital submodule mounted in this repository, to every future submodule, and to every code module added to a submodule. A submodule constitution MAY add stricter rules (e.g. Tracker-SDK's "no domain shape" rule) but MUST NOT relax 6.A–6.F. Adopting an externally maintained dependency that does not satisfy these clauses requires forking it under `vasic-digital/` first.

9.2 Cascaded scoped CLAUDE.md updates

File	Addition
core/CLAUDE.md	Reference 6.E. Add: every new feature interface added under <code>core/tracker/api/</code> MUST be accompanied by (a) a capability enum entry, (b) a behavioral test in <code>core/tracker/{rutracker,rutor}/test/</code> that exercises the interface end-to-end, (c) a mention in <code>docs/sdk-developer-guide.md</code> .
feature/CLAUDE.md	Add: every feature ViewModel that consumes <code>LavaTrackerSdk</code> MUST have a Challenge Test covering the same UI path. The Challenge Test's falsifiability rehearsal MUST be recorded in the same PR that introduces the ViewModel change.
lava-api-go/CLAUDE.md	Add SP-3a-bridge expectations: when SP-2 ships, the Go-side <code>rutracker</code> refactor MUST satisfy 6.D and 6.E.
lava-api-go/AGENTS.md	Mirror the above plus pointer to the Go-side bridge plan once written.
	Inherit Lava root + add "no domain shape" rule with named CI gate.

File	Addition
submodules/ tracker_sdk/ CLAUDE.md (new)	
submodules/ tracker_sdk/ CONSTITUTION.md (new)	Same content as CLAUDE.md, formatted as a constitution doc per project convention.
submodules/ tracker_sdk/ AGENTS.md (new)	Submodule-specific agent guide: how to add a new generic primitive, how to run the falsifiability rehearsal, where the four-upstream mirror policy lives.
Root AGENTS.md	Extend the agent guide with the new SDK module map, the Tracker-SDK pin policy, and pointers to the new Challenge Test pack location.

9.3 Local-Only CI/CD constraint reaffirmation

The pre-push hook, the tag gate, and scripts/ci.sh introduced or extended by SP-3a all run **locally only**. No .github/workflows/*, no GitLab CI, no GitFlic CI, no GitVerse CI files are added at any phase. Pre-push hook itself enforces this via a forbidden-paths check (per existing constitution).

10. Implementation Phases (high-level shape)

Detailed task/sub-task breakdown is the writing-plans skill's output. This section gives the phase shape; the plan doc fleshes it out.

Phase	Name	Duration	Key deliverables
0	Pre-flight (Task 1.0)	0.5w	Bluff audit of all fakes touched by SP-3a. Evidence pack. Exemption ledger seeded. vasic-digital/Tracker-SDK repo created on all four upstreams + mirrored. Submodule pinned at submodules/ tracker_sdk/. :core:tracker:api,:core:tracker
1	Foundation	2w	(thin wrapper over Tracker-SDK), :core:tracker:mirror (thin wrapper), :core:tracker:testing. New

Phase	Name	Duration	Key deliverables
2	RuTracker decoupling (Kotlin only)	2.5w	convention plugin <code>lava.kotlin.tracker.module</code> in <code>buildSrc/</code>. Constitutional clauses 6.D/6.E/6.F drafted. <code>git mv core/network/rutrack</code> <code>er → core/tracker/rutrack</code> <code>er.RuTrackerClient</code> implements <code>TrackerClient</code> + applicable feature interfaces. DTO [?] model mappers. Registry registration. <code>SwitchingNetworkApi</code> rewired to delegate to <code>LavaTrackerSdk</code>. Parity gate: byte-for-byte identical output for all 15 <code>NetworkApi</code> methods against pre-SP-3a baseline. No Go-side changes.
3	RuTor implementation	2w	<code>:core:tracker:rutor</code> module. All parsers with ≥5 fixtures each. <code>RuTorClient</code>. <code>RuTorAuthenticator</code>. Registry registration. Real-tracker integration test.
4	Mirror health + cross-tracker fallback + tracker_settings UI	1w	<code>WorkManager</code> periodic worker for health probes. Health state persisted to Room. Fallback chain executor with falsifiability rehearsal. Cross-tracker fallback policy + <code>CrossTrackerFallbackProposed</code> outcome + UI side effect. <code>:feature:tracker_settings</code> Compose screen (tracker selection + custom mirrors UI).
5	Constitutional updates + Challenge Tests + tag gate	0.5w	All 8 cascaded CLAUDE.md / AGENTS.md / CONSTITUTION.md updates. 8 Challenge Tests written, recorded, falsifiability-rehearsed. <code>scripts/ci.sh</code> and <code>scripts/tag.sh</code> updated with new gates. Release tag for Android 1.2.0.

Phase	Name	Duration	Key deliverables
(Bridge)	SP-3a-bridge — Go-side	1.5w (post-SP-2)	Go <code>tracker.TrackerClient</code> interface. Go <code>RuTrackerClient</code> . Go-Kotlin parity tests. OpenAPI updates. Constitutional updates in <code>lava-api-go/</code> .

Total SP-3a (excluding bridge): 8.5 weeks. The bridge runs after SP-2's last release tag and before SP-3b begins.

11. Versioning, Rollout, and Rollback

11.1 Version bumps after SP-3a ships

- **Android (:app):** 1.1.4 → 1.2.0. Semver minor — RuTor support is user-visible.
- **lava-api-go:** 2.0.7 → 2.0.8. No Go-side changes in SP-3a; this bump is documentation-only (CHANGELOG noting "client introduces multi-tracker SDK; Go side unchanged"). The next material Go bump is at SP-3a-bridge completion.
- **vasic-digital/Tracker-SDK:** 0.1.0 initial release at end of Phase 1.

11.2 Rollout

Standard release: build artifacts via `./build_and_release.sh`, sign with the existing keystore, push APK to the four upstreams' Releases pages, push the new Tracker-SDK repo to all four upstreams, advertise the new version in the existing release-notes channel. No staged rollout, no feature flag — RuTor is fully on or fully off and gated by the user's tracker selection in `:feature:tracker_settings`.

11.3 Rollback

- **Pre-Phase-2-Task-2.6:** new modules are additive. Reverting any new module is safe.
- **Post-Phase-2-Task-2.6:** rollback is a single revert of the `SwitchingNetworkApi` rewire commit. The new tracker modules can stay in place — the wire underneath is what flipped, and unwiring restores the pre-SP-3a code path completely. Feature ViewModels never knew the wire changed.
- **Post-tag (1.2.0 shipped):** users on 1.2.0 stay on 1.2.0; the rollback path is "ship 1.2.1 with the wire reverted." The tracker-settings UI tolerates

"only RuTracker is registered" gracefully (the registry just shows one option).

- **SP-3a-bridge rollback:** the Go side has its own rollback contract — `lava-api-go` redeployment from previous tag.

11.4 Pre-tag verification (Sixth Law clause 5)

Tag for 1.2.0 is **only** cut after a human (or a documented black-box runner) has used each Challenge-Tested feature on a real Android device against real RuTracker and real RuTor instances, and recorded the user-visible outcome. The verification record lives at `.lava-ci-evidence/Lava-Android-1.2.0-1020/real-device-verification.md`.

12. Risks & Mitigations

Risk	Likelihood	Mitigation
RuTor changes its HTML structure mid-implementation	Medium	Content-based selectors (Jackett pattern) tolerate column drift. Fixture freshness gate catches drift before tag.
RuTor blocks the IP / rate-limits / introduces captcha	Medium	Per-tracker rate limit semaphore (4 concurrent default). Real-tracker smoke tests detect block. Fall back to mirror with different IP via the existing mirror chain.
Cross-tracker fallback proposes RuTor for an authenticated RuTracker operation when user has no RuTor session	Medium	LavaTrackerSdk checks descriptor capabilities + <code>checkAuth()</code> state for the proposed alternate before emitting <code>CrossTrackerFallbackProposed</code> . If the proposed tracker would need login the user hasn't done, modal text changes to "Try RuTor — login required" and the user sees the consent friction explicitly.
The new submodule's CI gate produces friction for first-time contributors	Low	The submodule README documents <code>scripts/dev-setup.sh</code> that installs the pre-push hook and tooling in one command. Same pattern as Lava root.
Parity gate (Phase 2) detects byte-level drift the team can't explain	Medium	Phase 2 acceptance gate: byte-for-byte identical output. If divergence is found, root-cause it before tagging.

Risk	Likelihood	Mitigation
		Common cause is mapper rounding (e.g., timestamp truncation) — fix by adjusting mapper, not by relaxing the gate.
<code>:feature:tracker_settings</code> Compose UI is the largest new UI surface and is the most likely to ship with subtle bugs	Medium	Two Challenge Tests (C1 + C7) cover the screen end-to-end. Manual real-device verification per Sixth Law clause 5 before tag. Add to the Spec 2 Phase 6 mutation-test pass for re-verification.
Mirror health probes consume battery / data on metered networks	Low	WorkManager constraint: <code>setRequiredNetworkType(CONNECTED)</code> (not <code>UNMETERED</code>). Probe is GET / with 5s timeout — minimal data. User can disable periodic probing in <code>:feature:tracker_settings</code> .
The Tracker-SDK contract turns out to be wrong shape after RuTor exposes a behavior we didn't anticipate	Medium	Hybrid extraction (decision 2-C) keeps the tracker contract in this repo, not in the submodule, precisely so we can iterate cheaply without a submodule version bump. Re-evaluation point after SP-3a ships: does the contract survive RuTor unchanged?

13. Open Items (handled in writing-plans, not here)

The implementation plan that follows this spec must produce per-task and per-sub-task detail at the granularity the user requested in the brief. Specifically the plan should enumerate:

1. Each module's `build.gradle.kts` content and which convention plugin it applies.
2. Each parser's per-fixture assertions (which fields must be non-null, regex shapes for ID/hash fields, etc.).
3. Each mapper's field-by-field source/target with explicit metadata keys for tracker-specific extras.
4. The Room schema migration for `tracker_mirror_health` and `tracker_mirror_user` tables (new `core/database/schemas/<version>.json`).

5. Hilt DI module changes — which @Provides need updating and which new @Module are introduced.
 6. Each Challenge Test's Compose UI selectors and assertion screenshots.
 7. The scripts/ci.sh and scripts/tag.sh change-set in full.
 8. The four-upstream sync sequence for vasic-digital/Tracker-SDK initial creation.
 9. Each constitutional doc's exact diff (so reviewers can see precisely what's added).
 10. The CHANGELOG.md entry for Lava 1.2.0.
 11. The SP-3a-bridge plan's phase shape (the bridge has its own writing-plans output later, but the spec must declare its boundaries here).
-

14. Success Criteria (binding for SP-3a tag)

SP-3a is "done" when **all** of the following are true:

1. submodules/tracker_sdk/ exists, is mirrored to all four upstreams, has its own constitution + CI evidence, and is pinned at a frozen hash in this repo.
 2. :core:tracker:api, :core:tracker:client, :core:tracker:ruTracker, :core:tracker:ru are all present and applied.
 3. :core:network:ruTracker is removed from settings.gradle.kts; its history is preserved at :core:tracker:ruTracker via git mv.
 4. SwitchingNetworkApi delegates to LavaTrackerSdk. Parity gate passes byte-for-byte.
 5. Eight Challenge Tests pass on a real Android device against real RuTracker and real RuTor instances, with screenshots in the evidence pack.
 6. Coverage gates met: ≥95% / ≥90% / ≥85% line, ≥85% mutation kill, behavioral coverage exemption ledger committed.
 7. All eight constitutional doc updates landed, pre-push hook + tag-gate enforce the new clauses.
 8. Lava-Android-1.2.0-1020 tag cut, mirrored to all four upstreams with verified per-mirror SHA convergence.
 9. The decision log in §0 of this spec matches the decisions actually implemented.
 10. docs/sdk-developer-guide.md (Phase 7 deliverable in Spec 2 — partial draft acceptable for SP-3a) outlines the steps for adding a third tracker, validated by a paper trace through the existing RuTor module.
-

End of design.